```cpp
const int N = 1e4 + 10, M = 1010;

int n, m;
int t[N], c[N], p[N];
int f[M];

int main()
{
    int t1, t2, t3, t4; char ch;
    cin >> t1 >> ch >> t2 >> t3 >> ch >> t4 >> n;
    m = t3 * 60 + t4 - (t1 * 60 + t2);

    for(int i = 1; i <= n; i++) cin >> t[i] >> c[i] >> p[i];

    for(int i = 1; i <= n; i++)
    {
        if(p[i] == 0) // 完全背包
        {
            for(int j = t[i]; j <= m; j++)
            {
                f[j] = max(f[j], f[j - t[i]] + c[i]);
            }
        }
        else // 多重背包 或 01 背包
        {
            for(int j = m; j >= t[i]; j--)
            {
                for(int k = 1; k <= p[i] && k * t[i] <= j; k++)
                {
                    f[j] = max(f[j], f[j - t[i] * k] + c[i] * k);
                }
            }
        }
    }

    cout << f[m] << endl;

    return 0;
}
```

3.6 多维费用的背包问题

3.6.1 L 国的战斗之间谍

> **题目来源：** 洛谷
>
> **题目链接：** P1910 L 国的战斗之间谍
>
> **难度系数：** ★★

【题目描述】

> 俗话说的好：“知己知彼，百战不殆”。L 国的指挥官想派出间谍前往I国，于是，选人工作就落到了你身上。
>
> 你现在有 N 个人选，每个人都有这样一些数据： A （能得到多少资料）、 B （伪装能力有多差）、 C （要多少工资）。已知敌人的探查间谍能力为 M （即去的所有人 B 的和要小于等于 M ）和手头有 X 元钱，请问能拿到多少资料？

【输入描述】

> 第一行三个整数 N, M, X 代表总人数，敌国侦察能力和总钱数。
>
> 第二行至第 $N+1$ 行，每行三个整数 A_i, B_i, C_i 分别表示第 i 个人能得到的资料，他的伪装能力有多差和他要的工资。
>
> 数据范围： $1 \leq n \leq 100, 1 \leq m \leq 1000, 1 \leq x \leq 1000$。

【输出描述】

> 一行一个整数表示能得到的资料总数。

【示例一】

> 输入：
>
> ```
> 3 10 12
> 10 1 11
> 1 9 1
> 7 10 12
> ```
>
> 输出：
>
> ```
> 11
> ```

【解法】

无非就是在 01 背包的基础上多加了一维，那我们就把状态表示也加上一维即可。

1. 状态表示：

   $dp[i][j][k]$ 表示：从前 i 个人中挑选，伪装能力之和不超过 j ，总工资不超过 k ，此时能获取到的最多资料总数。

   那么 $dp[n][m][x]$ 就是结果。

2. 状态转移方程：

   根据第 i 个人选或者不选分两种情况讨论：

   a. 不选：此时的最多资料为 $dp[i-1][j][k]$；

   b. 选：那就要去前 $i-1$ 各种，凑伪装能力之和不超过 $j-b[i]$，总工资不超过 $k-c[i]$ 时的最多工资，再加上第 i 个人的工资。也就是 $dp[i-1][j-b[i]][k-c[i]]+a[i]$。

   取上述两种情况的最大值即可。注意第二种情况要特判一下。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110, M = 1010;

int n, m, x;
int a[N], b[N], c[N];
int f[M][M];

int main()
{
    cin >> n >> m >> x;
    for(int i = 1; i <= n; i++) cin >> a[i] >> b[i] >> c[i];

    for(int i = 1; i <= n; i++)
        for(int j = m; j >= b[i]; j--)
            for(int k = x; k >= c[i]; k--)
                f[j][k] = max(f[j][k], f[j - b[i]][k - c[i]] + a[i]);

    cout << f[m][x] << endl;


    return 0;
}
```

4. 区间 dp

区间 dp 也是线性 dp 的一种，它用区间的左右端点来描述状态，通过小区间的解来推导出大区间的解。因此，区间 DP 的核心思想是将大区间划分为小区间，它的状态转移方程通常依赖于区间的划分点。

常用的划分点的方式有两个：

- 基于区间的左右端点，分情况讨论；
- 基于区间上某一点，划分成左右区间讨论。

4.1 回文字串

> **题目来源：** 洛谷
>
> **题目链接：** P1435 [IOI2000] 回文字串
>
> **难度系数：** ★★

【题目描述】

> 回文词是一种对称的字符串。任意给定一个字符串，通过插入若干字符，都可以变成回文词。此题的任务是，求出将给定字符串变成回文词所需要插入的最少字符数。
>
> 比如 $Ab3bd$ 插入 2 个字符后可以变成回文词 $dAb3bAd$ 或 $Adb3bdA$，但是插入少于 2 个的字符无法变成回文词。
>
> **注意**：此问题区分大小写。

【输入描述】

> 输入共一行，一个字符串。
>
> 对于全部数据，$0 < l \leq 1000$。

【输出描述】

> 有且只有一个整数，即最少插入字符数。

【示例一】

> 输入：
>
> Ab3bd
>
> 输出：
>
> 2

【解法】

先找重复子问题定义状态表示：

- 大问题是让整个字符串 $[1,n]$ 变成回文串的最小插入次数；
- 当我们发现这个字符串左右元素一样的时候，那就去看看 $[2,n-1]$ 变成回文的最小插入次数；
- 如果左右不相同，那么我们会在左边补上一个字符，或者右边补上一个字符，然后看看剩下区间的最小插入次数。

因此，重复的子问题就是看看某个区间变成回文串的最小插入次数。

1. 状态表示：

   $dp[i][j]$ 表示：字符串 $[i,j]$ 区间，变成回文串的最小插入次数。

   那么 $dp[1][n]$ 就是我们要的结果。

2. 状态转移方程：

   根据区间的左右端点，分情况讨论：

   a. 如果 $s[i]=s[j]$：那我们就去看看 $[i+1,j-1]$ 区间的最小插入次数，即 $dp[i+1][j-1]$；

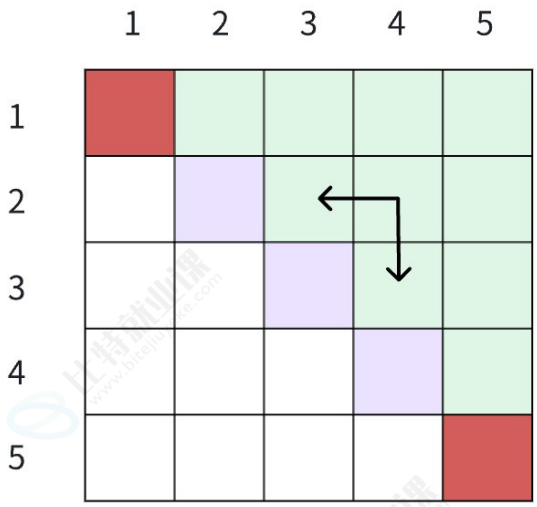
   b. 如果 $s[i]\neq s[j]$：

   - 要么去左边补一个 $s[j]$，此时的最小插入次数为 $dp[i][j-1]+1$；
   - 要么去右边补一个 $s[i]$，此时的最小插入次数为 $dp[i+1][j]+1$。

   因为要的是最小值，所以状态转移方程为 $min(dp[i+1][j],dp[i][j-1])+1$。

3. 初始化以及填表顺序：

我们看 dp 表：



可以发现如下性质：

- 白色部分是用不到的非法区域，因为这个区域中的左端点大于右端点，不符合区间的定义；
- 每一个格子填表的时候，需要左边的格子以及下边的格子；
- 当 $i = j$ 的时候，填格子会用到非法区域，并且 $i = 1$ 以及 $i = n$ 的时候会越界，需要特殊处理。

综上所述：

- 对于初始化：我们需要初始化对角线位置的值。因为对角线表示长度为 1 的字符串，本身就是回文串，里面的值是 0 即可。
- 对于填表顺序，我们有两种策略：
  a. 从下往上填写每一行，每一行从左往右。这样就能保证在填写 $[i, j]$ 位置时，$[i + 1, j]$ 以及 $[i, j - 1]$ 已经被更新过了；
  b. 第一维循环：小到大枚举区间长度 $len(2 \le len \le n)$；第二维循环：枚举区间左端点 i ；然后计算出区间右端点 $j = i + len - 1$。这样我们填表的时候，就是一个对角线一个对角线的填，不会产生越界访问的问题。

对于区间 dp 的填表顺序，我们一般选取第二种，会让我们的代码看着很清晰，也比较符合区间 dp 的推导过程，从小区间递推到大区间。

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 1010;

int f[N][N];

int main()
{
    string s; cin >> s;
    int n = s.size();
    s = " " + s;

    for(int len = 1; len <= n; len++) // 枚举长度
        for(int i = 1; i + len - 1 <= n; i++) // 枚举左端点
        {
            int j = i + len - 1;
```

```
19
20              if(s[i] == s[j]) f[i][j] = f[i + 1][j - 1];
21              else f[i][j] = min(f[i + 1][j], f[i][j - 1]) + 1;
22          }
23
24      cout << f[1][n] << endl;
25
26      return 0;
27  }
```

4.2 Treats for the Cows

题目来源： 洛谷

题目链接： P2858 [USACO06FEB] Treats for the Cows G/S

难度系数： ★★

【题目描述】

约翰经常给产奶量高的奶牛发特殊津贴，于是很快奶牛们拥有了大笔不知该怎么花的钱。为此，约翰购置了 $N(1 \leq N \leq 2000)$ 份美味的零食来卖给奶牛们。每天约翰售出一份零食。当然约翰希望这些零食全部售出后能得到最大的收益，这些零食有以下这些有趣的特性：

- 零食按照 $1, \ldots, N$ 编号，它们被排成一列放在一个很长的盒子里。盒子的两端都有开口，约翰每天可以从盒子的任一端取出最外面的一个。
- 与美酒与好吃的奶酪相似，这些零食储存得越久就越好吃。当然，这样约翰就可以把它们卖出更高的价钱。
- 每份零食的初始价值不一定相同。约翰进货时，第 i 份零食的初始价值为 $V_i(1 \leq V \leq 1000)$ 。
- 第 i 份零食如果在被买进后的第 a 天出售，则它的售价是 $V_i \times a$ 。

V_i 的是从盒子顶端往下的第i份零食的初始价值。约翰告诉了你所有零食的初始价值，并希望你能帮他计算一下，在这些零食全被卖出后，他最多能得到多少钱。

【输入描述】

第一行：整数 N ；

接下来 N 行，分别表示 V_i 。

【输出描述】

一个整数，表示能得到的最大价钱。

【示例一】

> 输入：
>
> 5
>
> 1
>
> 3
>
> 1
>
> 5
>
> 2
>
> 输出：
>
> 43

【解法】

贪心：每次都拿两边最小的。反例：$4,1,5,3$。

- 贪心解：$3\times1+4\times2+1\times3+5\times4=34$
- 正解：$4\times1+1\times2+3\times3+5\times4=35$

原因是，鼠目寸光。看似当前把最小的拿走了，但是如果先拿走一个较大的，可能会把更小的暴露出来。

正解还是老老实实的区间 dp：

1. 状态表示：

   $dp[i][j]$ 表示：把区间 $[i,j]$ 的零食全部拿走，最多能得到多少钱。

2. 状态转移方程：

   根据先拿左边还是先拿右边，能分成两种情况讨论：

   a. 先拿左边，然后去 $[i+1,j]$ 区间获得最多的钱，即 $a[i]\times(n-len+1)+dp[i+1][j]$；

   b. 先拿右边，然后去 $[i,j-1]$ 区间获得最多的钱，即 $a[j]\times(n-len+1)+dp[i][j-1]$；

   因为要的是最多的钱，所以应该是上面两种情况的最大值。

3. 初始化：

   当区间长度为 1 时：$dp[i][i]=n\times a[i]$。要注意，长度为 1，那就是第 n 次拿。

4. 填表顺序：

   先枚举区间长度，再枚举左端点，右端点通过计算。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 2010;

int n;
int a[N];
int f[N][N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    for(int len = 1; len <= n; len++)
        for(int i = 1; i + len - 1 <= n; i++)
        {
            int j = i + len - 1;
            int cnt = n - len + 1;

            f[i][j] = max(f[i + 1][j] + a[i] * cnt, f[i][j - 1] + a[j] * cnt);
        }

    cout << f[1][n] << endl;

    return 0;
}
```

4.3 石子合并（弱化版）

题目来源： 洛谷

题目链接： P1775 石子合并（弱化版）

难度系数： ★★

【题目描述】

设有 $N(N \leq 300)$ 堆石子排成一排，其编号为 $1, 2, 3, \cdots, N$。每堆石子有一定的质量 $m_i(m_i \leq 1000)$。现在要将这 N 堆石子合并成为一堆。每次只能合并相邻的两堆，合并的代价为

> 这两堆石子的质量之和，合并后与这两堆石子相邻的石子将和新堆相邻。合并时由于选择的顺序不同，合并的总代价也不相同。试找出一种合理的方法，使总的代价最小，并输出最小代价。

【输入描述】

> 第一行，一个整数 N。
>
> 第二行，N 个整数 m_i。

【输出描述】

> 输出文件仅一个整数，也就是最小代价。

【示例一】

> 输入：
>
> 4
>
> 2 5 3 1
>
> 输出：
>
> 22

【解法】

1. 状态表示：

   $dp[i][j]$ 表示：合并区间 $[i,j]$ 石子，最小的代价。

   那么 $dp[1][n]$ 就是结果

2. 状态转移方程：

   根据最后一步合并的情况，可以分成 $j-i$ 种情况。设最后一步合并的时候，两个区间的分割点为 k，也就是区间被分成 $[i,k]$ 和 $[k+1,j]$，此时的最小代价为合并左边区间的最小代码 + 合并右边区间的最小代价 + 合并两个区间的代价，即
   $dp[i][k]+dp[k+1][j]+sum[i,k]+sum[k+1,j]$。

   其中 $sum[i,k]+sum[k+1,j]$ 其实就是整个区间的和，可以用前缀和数组预处理一下，就可快速求出来。

   因为要的是最小代价，所以状态转移方程就是所有 k 变化范围内的最小值。

3. 初始化：

   区间长度为 1 的时候，不需要合并，代价为 0。

4. 填表顺序：

   先枚举区间长度，再枚举左端点，右端点通过计算。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 310;

int n;
int f[N][N];
int sum[N]; // 前缀和数组

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        sum[i] = sum[i - 1] + x;
    }

    // 初始化
    memset(f, 0x3f, sizeof f);
    for(int i = 0; i <= n; i++) f[i][i] = 0;

    for(int len = 2; len <= n; len++)
    {
        for(int i = 1; i + len - 1 <= n; i++)
        {
            int j = i + len - 1;
            int t = sum[j] - sum[i - 1];

            // 枚举分割点
            for(int k = i; k < j; k++)
            {
                // [i, k] [k + 1, j]
                f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + t);
            }
        }
    }

    cout << f[1][n] << endl;

    return 0;
```

```
44    }
```

4.4 石子合并

> **题目来源：** 洛谷
>
> **题目链接：** P1880 [NOI1995] 石子合并
>
> **难度系数：** ★★★

【题目描述】

> 在一个圆形操场的四周摆放 N 堆石子，现要将石子有次序地合并成一堆，规定每次只能选相邻的 2 堆合并成新的一堆，并将新的一堆的石子数，记为该次合并的得分。
>
> 试设计出一个算法,计算出将 N 堆石子合并成 1 堆的最小得分和最大得分。

【输入描述】

> 数据的第 1 行是正整数 N，表示有 N 堆石子。
>
> 第 2 行有 N 个整数，第 i 个整数 a_i 表示第 i 堆石子的个数。
>
> $1 \leq N \leq 100, 0 \leq a_i \leq 20$。

【输出描述】

> 输出共 2 行，第 1 行为最小得分，第 2 行为最大得分。

【示例一】

> 输入：
>
> ```
> 4
> 4 5 9 4
> ```
>
> 输出：
>
> ```
> 43
> 54
> ```

【解法】

处理环形问题的技巧：倍增。

在数组后面，将原始数组复写一遍，然后在倍增之后的数组上做一次石子合并（弱化版），就能得到以所有位置为起点并且长度为 len 的最小合并代价。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 210;

int n, m;
int s[N];
int f[N][N]; // 最小得分
int g[N][N]; // 最大得分

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> s[i];
        // 倍增
        s[i + n] = s[i];
    }
    m = n + n;

    // 前缀和
    for(int i = 1; i <= m; i++)
    {
        s[i] += s[i - 1];
    }

    // 初始化
    memset(f, 0x3f, sizeof f);
    memset(g, -0x3f, sizeof g);
    for(int i = 1; i <= m; i++)
    {
        f[i][i] = g[i][i] = 0;
    }

    for(int len = 1; len <= n; len++)
    {
        for(int i = 1; i + len - 1 <= m; i++)
        {
            int j = i + len - 1;
            int t = s[j] - s[i - 1];

            // 枚举分割点
            for(int k = i; k < j; k++)
```

```
            {
                // [i, k] [k + 1][j]
                f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + t);
                g[i][j] = max(g[i][j], g[i][k] + g[k + 1][j] + t);
            }
        }
    }

    // 更新结果
    int ret1 = 0x3f3f3f3f, ret2 = -0x3f3f3f3f;
    for(int i = 1; i <= n; i++)
    {
        ret1 = min(ret1, f[i][i + n - 1]);
        ret2 = max(ret2, g[i][i + n - 1]);
    }

    cout << ret1 << endl << ret2 << endl;

    return 0;
}
```

4.5 248

> **题目来源：** 洛谷
>
> **题目链接：** P3146 [USACO16OPEN] 248 G
>
> **难度系数：** ★★★

【题目描述】

> 贝西喜欢在手机上下载游戏来玩，尽管她确实觉得对于自己巨大的蹄子来说，小小的触摸屏用起来相当笨拙。
>
> 她对当前正在玩的这个游戏特别感兴趣。游戏开始时给定一个包含 N 个正整数的序列（ $2 \leq N \leq 248$ ），每个数的范围在 $1 \ldots 40$ 之间。在一次操作中，贝西可以选择两个相邻且相等的数，将它们替换为一个比原数大 1 的数（例如，她可以将两个相邻的 7 替换为一个 8 ）。游戏的目标是最大化最终序列中的最大数值。请帮助贝西获得尽可能高的分数！

【输入描述】

> 第一行包含整数 N ，接下来 N 行表示给定的游戏序列。

【输出描述】

> 输出能生成的最大整数。

【示例一】

输入：

4

1

1

1

2

输出：

3

【解法】

1. 状态表示：

   $dp[i][j]$ 表示：将区间 $[i,j]$ 合并的只剩下一个元素后，能得到的最大值。

   至于为什么要定义合并剩一个元素，因为如果不这样定义，相邻两个区间最大值虽然一样，但是不一定能合并。

   那么 dp 表里面的最大值就是结果，因为有些区间可能无法合并。

2. 状态转移方程：

   跟石子合并的讨论方式一样，根据最后一次合并的情况，可以把区间分成 $[i,k]$ 和 $[k+1,j]$，要想能够合并，需要满足下面条件：

   a. 两者合并后的最大值一致，才能合并：$dp[i][k]=dp[k+1][j]$；

   b. 合并后的最大值不能是 0。如果是 0，说明根本就不能合并：$dp[i][k]\neq 0$。

   如果能合并，合并后的最大值就是 $dp[i][k]+1$。那么状态转移方程就是所有符合要求的 k 里面的最大值。

3. 初始化：

   所有长度为 1 的区间，合并后的最大值应该是自己。所以初始化所有的 $dp[i][i]=a[i]$，也就是对角线。

4. 填表顺序：

   先枚举区间长度，再枚举左端点，右端点通过计算。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 255;

int n;
int a[N];
int f[N][N];

int main()
{
    cin >> n;
    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i];
        f[i][i] = a[i];
        ret = max(ret, a[i]);
    }

    for(int len = 2; len <= n; len++)
    {
        for(int i = 1; i + len - 1 <= n; i++)
        {
            int j = i + len - 1;

            // 枚举分割点
            for(int k = i; k < j; k++)
            {
                // [i, k] [k + 1, j]
                if(f[i][k] && f[i][k] == f[k + 1][j])
                {
                    f[i][j] = max(f[i][j], f[i][k] + 1);
                }
            }
            // 更新结果
            ret = max(ret, f[i][j]);
        }
    }

    cout << ret << endl;

    return 0;
}
```